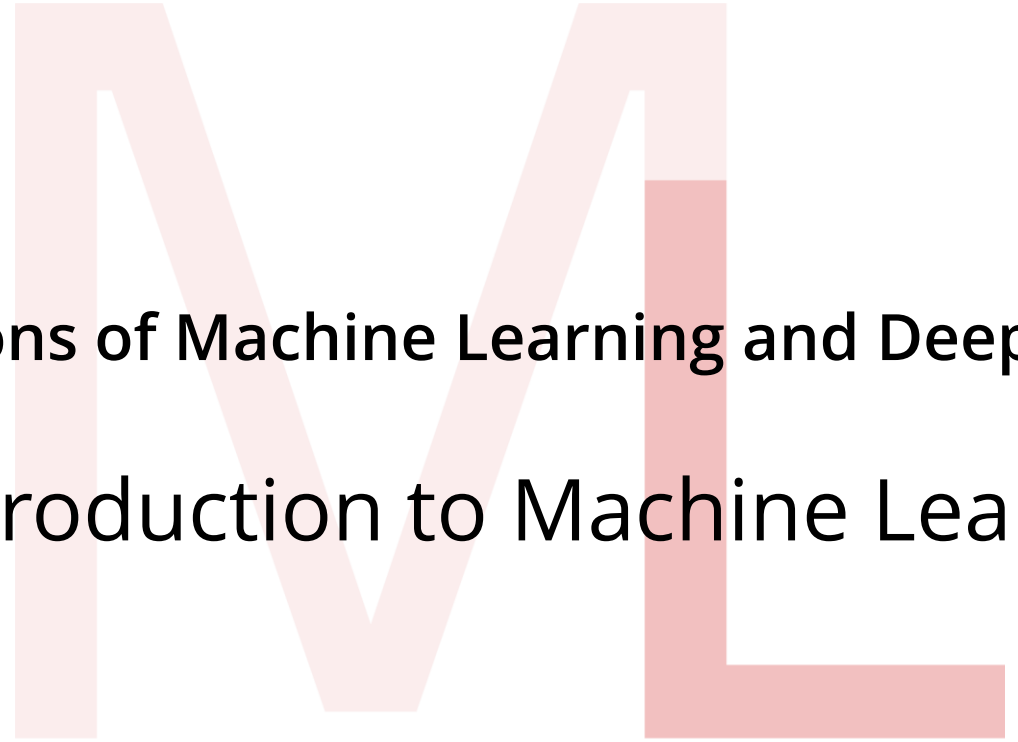
Foundations of Machine Learning and Deep Learning

1. Introduction to Machine Learning



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Outline

- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Classical Models – Ensembles

- An **ensemble model** combines the predictions of multiple base learners to produce a stronger overall model
  - Idea “the wisdom of the crowd”: Groups of weak models can outperform a single strong model

- Types of ensemble:
  - Homogeneous: Same base learner (e.g., decision tree), trained with randomness or reweighting
  - Heterogeneous: Different base learners (e.g., Naïve Bayes + kNN + decision tree) combined via meta-model

Classical Models – Ensembles

Motivation

- Limitations of a single model:
  - A very simple model may underfit (poor performance)
  - A very complex model may overfit (memorizes training data)
  - Some models are unstable (small changes in data lead to very different results)
- Ensemble idea:
  - Combining multiple “weak” learners can make a stronger one
  - The group prediction is often more accurate and more stable than any single model
- The key: **Diversity**

Classical Models – Ensembles

- **Diversity**
  - If all base learners are identical, combining them changes nothing
  - An ensemble works only if its members make different mistakes
  - Diversity among learners is **essential**

- **How to create diversity?**
  - Data sampling: train each learner on a different random subset of data
  - Feature sampling: train each learner on different subsets of features
  - Sequential reweighting: adjust sample weights to focus on hard cases

Classical Models – Ensembles

General algorithm

- Train T base estimators with diversity
  - Each with different data / features / weights
- Combine the results

Different types of ensemble models

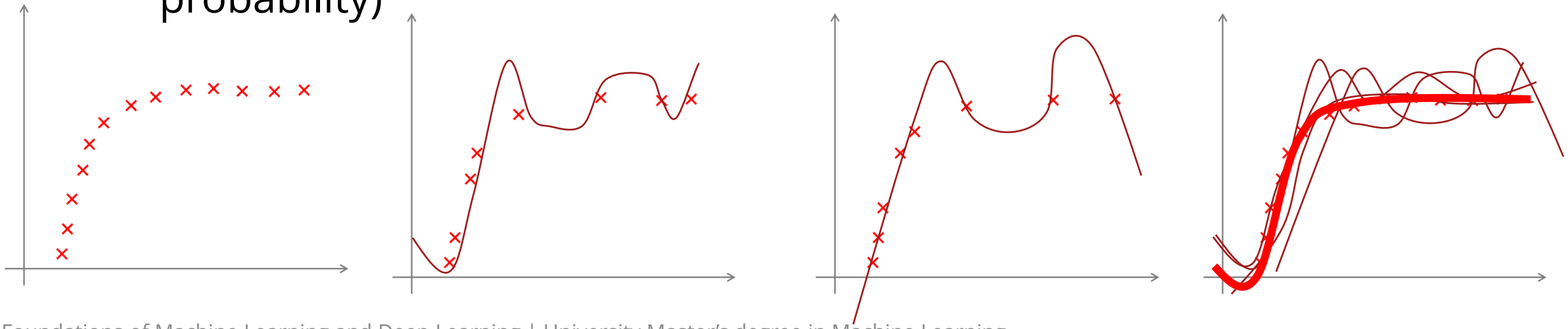
- Bagging (Bootstrap Aggregating) + Random Subpsace Method
- Random Forest
- Boosting
  - Adaboost
  - Gradient Boosting (XGBoost)

Classical Models – Ensembles

Bagging (Bootstrap Aggregating)

- Repeat T times (one per base estimator)
  - Retrieve a bag of data (bootstrap sampling) – random
  - Train a base estimator
- Output combination (classification)
  - Simple vote (each estimator votes 0/1 to the predicted label)
  - Weighted vote (each estimator votes according to the predicted probability)

It reduces overfitting!



Bagging (Bootstrap Aggregating)

Algorithm 1 Bagging

Input: S: Training set; T: Number of iterations;
n: Bootstrap size; I: Weak learner

Output: Bagged classifier: $H(x) = sign\left(\sum_{t=1}^{T} h_t(x)\right)$ where $h_t \in [-1, 1]$ are the induced classifiers

1: **for** $t = 1$ to T **do**
2: $S_t \leftarrow$ RandomSampleReplacement(n,S)
3: $h_t \leftarrow$ I(S_t)
4: **end for**

Classical Models – Ensembles

Random subspace method

- Repeat T times (one per base estimator)
  - Select a subset of p features (p < #features) – random
  - Train a base estimator
- Output combination (classification)
  - Simple vote (each estimator votes 0/1 to the predicted label)
  - Weighted vote (each estimator votes according to the predicted probability)
- Output combination (regression)
  - The outputs are averaged

- Bagging and Random Subspace model can be combined
  - E.g., Random Forest

Classical Models – Ensembles

Random Forest

- The base estimators are decision trees
- TRAINING IDEA:
  - Select the number of base estimators (T)
  - For each decision tree
    - Sample *N* examples via bootstrap
    - Train the decision tree with that data
      - But, in each node *p* features are selected ($p < \#features$)
      - The best out of those *p* features is selected with an heuristic
    - The tree is not pruned

Classical Models – Ensembles

Random Forest

- The base estimators are decision trees
- PREDICTION IDEA:
  - The new example if classified by all base estimators
  - Each tree votes the class
    - W.r.t. the prediction probabilities
  - The class with the highest mean probability wins

Classical Models – Ensembles

Random Forest

- Advantages
  - Good performance
  - Robust against outliers
  - Fast and simple
- Disadvantages
  - Less interpretable than a tree

Classical Models – Ensembles

Boosting

- Build an ensemble sequentially
  - Each new estimator focuses on the mistakes of the previous ones
- Final model: a weighted combination of many weak learners
- Usually, base learners are very simple models (e.g., decision stumps)
- Boosting reduces underfitting by making weak learners cooperate

- **Two main methods:**
  - AdaBoost: reweighting misclassified samples
  - Gradient Boosting: fitting new learners to residuals of the loss function

Classical Models – Ensembles

AdaBoost

- Important: the base estimator must be able to handle weights
- General algorithm
  - Assign the same weight to each sample
  - Repeat T times
    - Learn a weak classifier with current weights
    - Modify the weights
      - Failed samples should have more weight
      - Correctly classified examples should have less weight
    - Assign a weight to the current base classifier

AdaBoost

Algorithm 3 AdaBoost

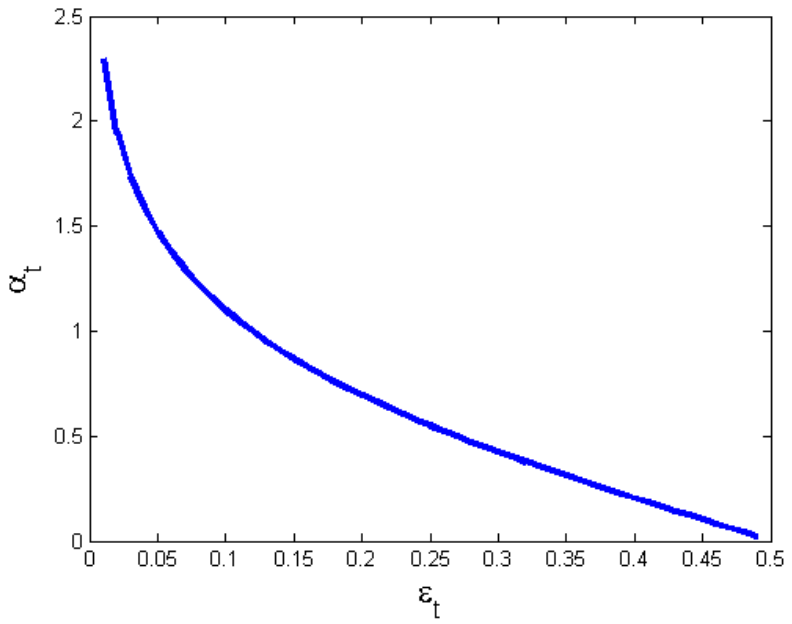
Input: Training set $S = \{\mathbf{x}_i, y_i\}$, $i = 1, \ldots, N$; and $y_i \in \{-1, +1\}$; T: Number of iterations; I: Weak learner

Output: Boosted classifier: $H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$ where h_t, α_t are the induced classifiers (with $h_t(x) \in \{-1, 1\}$) and their assigned weights, respectively

1: $D_1(i) \leftarrow 1/N$ for $i = 1, \ldots, N$
2: **for** $t = 1$ to T **do**
3: $\quad h_t \leftarrow I(S, D_t)$
4: $\quad \varepsilon_t \leftarrow \sum_{i, y_i \neq h_t(\mathbf{x}_i)} D_t(i)$
5: $\quad$ **if** $\varepsilon_t > 0.5$ **then**
6: $\quad\quad T \leftarrow t - 1$
7: $\quad\quad$ **return**
8: $\quad$ **end if**
9: $\quad \alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$
10: $\quad D_{t+1}(i) = D_t(i) \cdot e^{(-\alpha_t h_t(\mathbf{x}_i) y_i)}$ for $i = 1, \ldots, N$
11: $\quad$ Normalize D_{t+1} to be a proper distribution
12: **end for**

$$\varepsilon_t \leftarrow \sum_{i, y_i \neq h_t(\mathbf{x}_i)} D_t(i)$$

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$$



Gradient Boosting

- Build an ensemble additively
  - The idea is to get a series of sequential estimators
  - It starts from a simple estimator and incorporates values to compensate for the errors of the previous estimator
- Works for both regression and classification
- More flexible than AdaBoost: any differentiable loss can be used
  - MSE is typical
- Optimized in the XGBoost library

Gradient Boosting - idea

- Start from a constant estimator $$F_0(\mathbf{x}) = \arg\min_{c \in \mathbb{R}} \sum_{i=1}^{n} \ell(y_i, c)$$

- The predictions of in-between regression models are added to the estimator
  - These regressors are fitted to the **residuals**

The **residuals** are the difference between the real value of the target variable and the prediction for each sample

$$r_i = y_i - \hat{y}_i$$

Gradient Boosting - idea

- The model is additive: terms are added

$$\hat{y}_i = F_T(x_i) = \sum_{t=1}^{T} h_t(x_i)$$

  - T is the number of base estimators
  - h_t is each base estimator (normally decision trees)
- Then, intermediate results are constru

$$F_t(x) = F_{t-1}(x) + h_t(x)$$

  - where

$$h_t = \arg\min_h L_t = \arg\min_h \sum_{i=1}^{n} l(y_i, F_{t-1}(x_i) + h(x_i))$$

$l(y_i, F(x_i))$ is the differentiable loss

Gradient Boosting - idea

- Using Taylor approximation

$$h_t \approx \arg\min_h \sum_{i=1}^{n} h(x_i) \left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}$$

- To find a function *h* that minimizes the expression, a regression model (*h*) is fitted to the target variable given by

$$-\left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}$$

  - Which is the residual (whenever *l is MSE)*

- Finally, a shrinkage parameter is include $F_t(x) = F_{t-1}(x) + \nu h_t(x)$

Gradient Boosting – algorithm

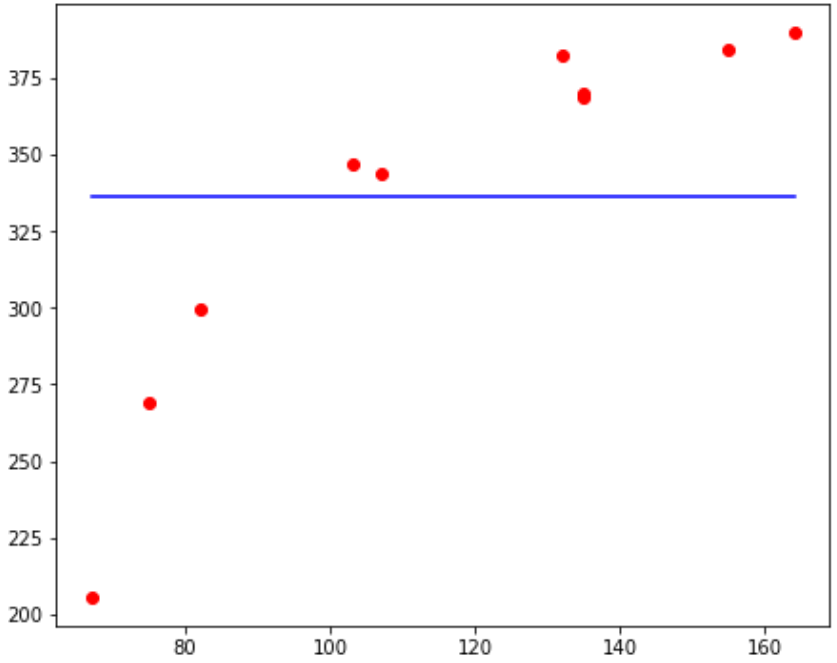
Given: Dataset (X, y) , a shrinkage parameter and a number of iterations T

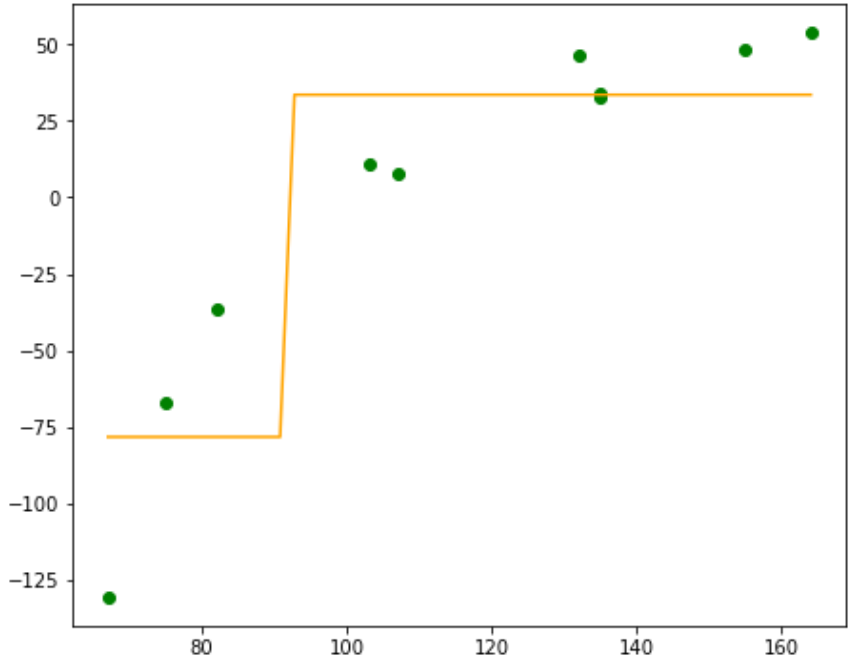
1. Initialize the model $$F_0(x) = \arg\min_{w \in \mathbb{R}} \sum_{i=1}^{N} \ell(y_i, w)$$

2. For t = 1, …, T
   a) Compute residuals $$r_{t-1}(x_i) = -\left[\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{t-1}}$$ *(for squared loss: $r_{t-1} = y - F_{t-1}(X)$)*
   b) Fit base learner $h_t(x)$ to predict r_{t-1}
   c) Update model $F_t(x) = F_{t-1}(x) + \nu h_t(x)$

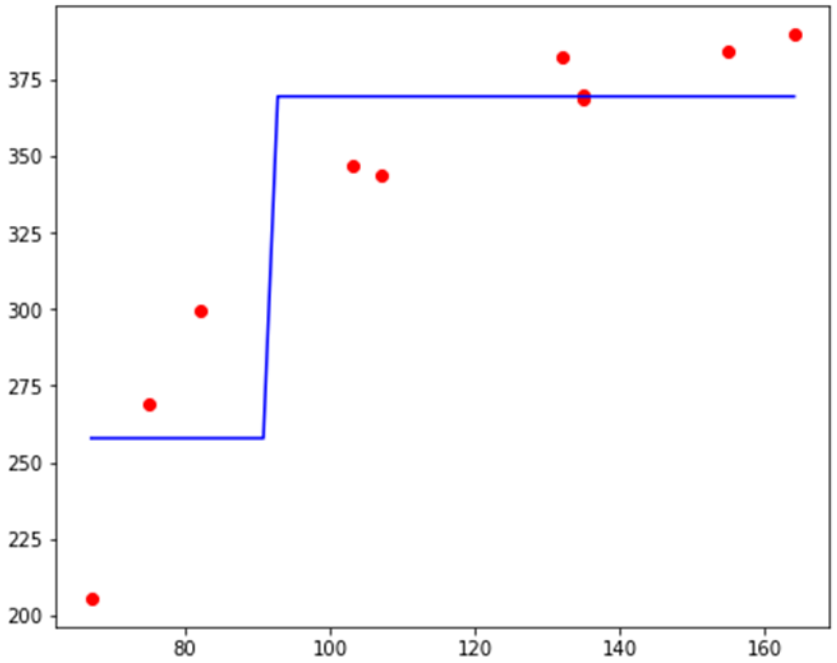
3. Final predictor: $F_T(x)$

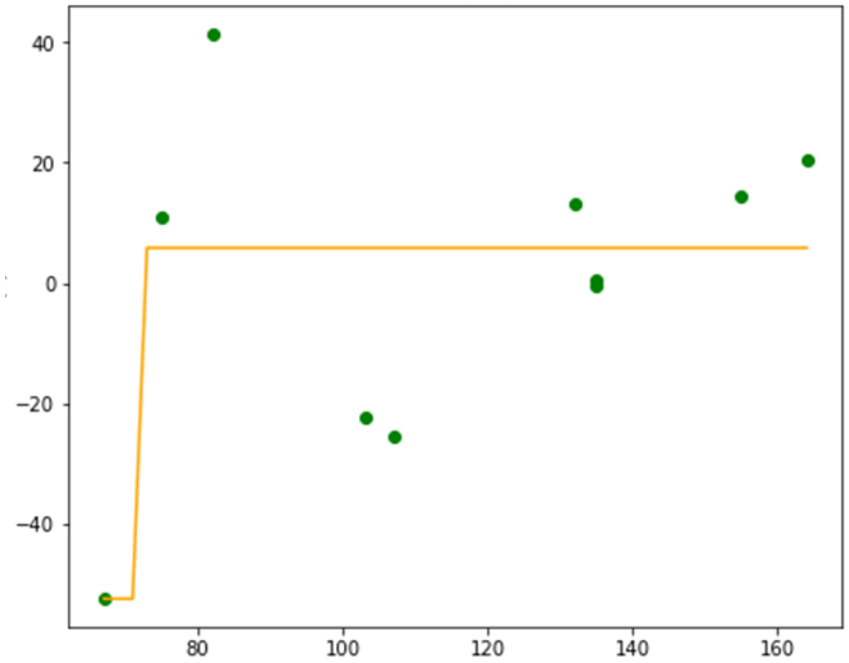
Gradient Boosting – example



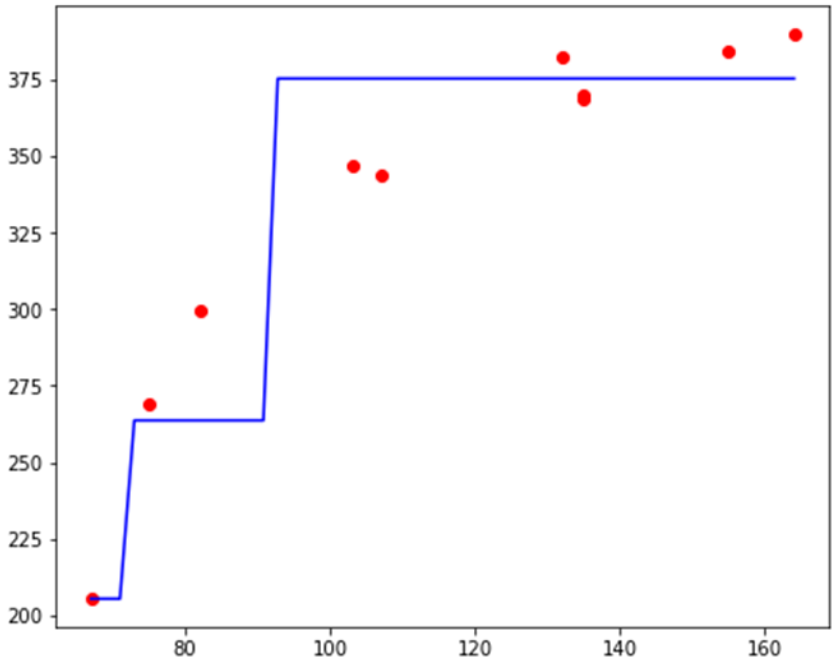


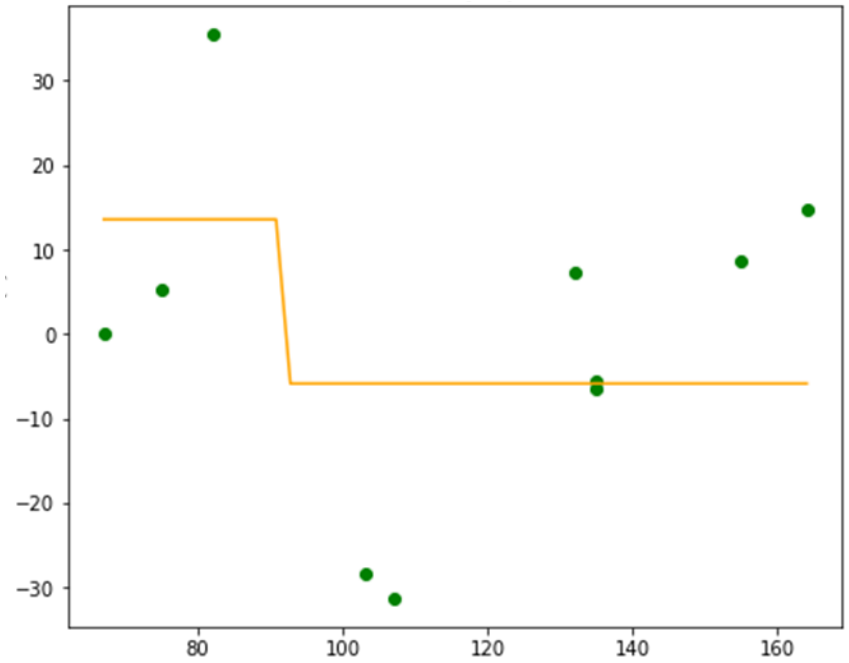
Gradient Boosting – example



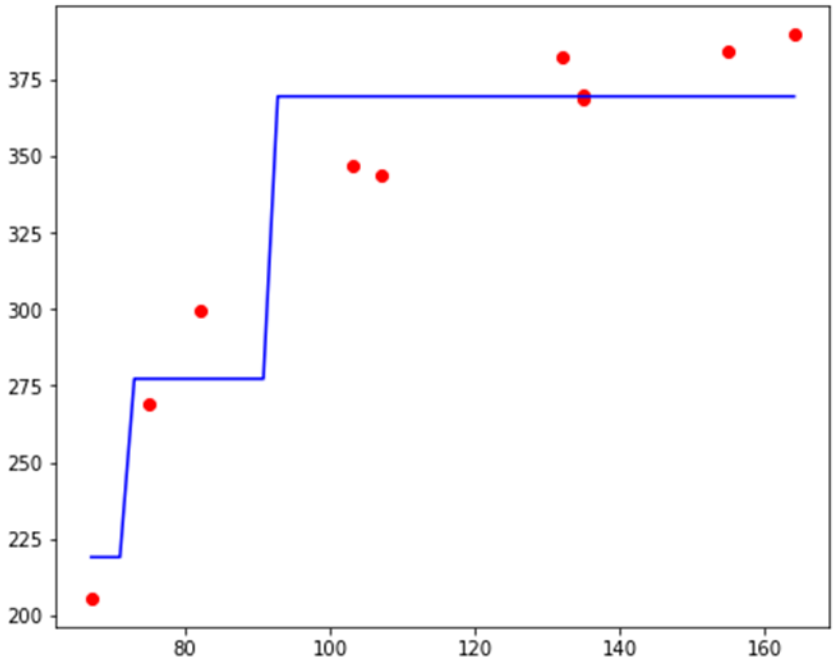


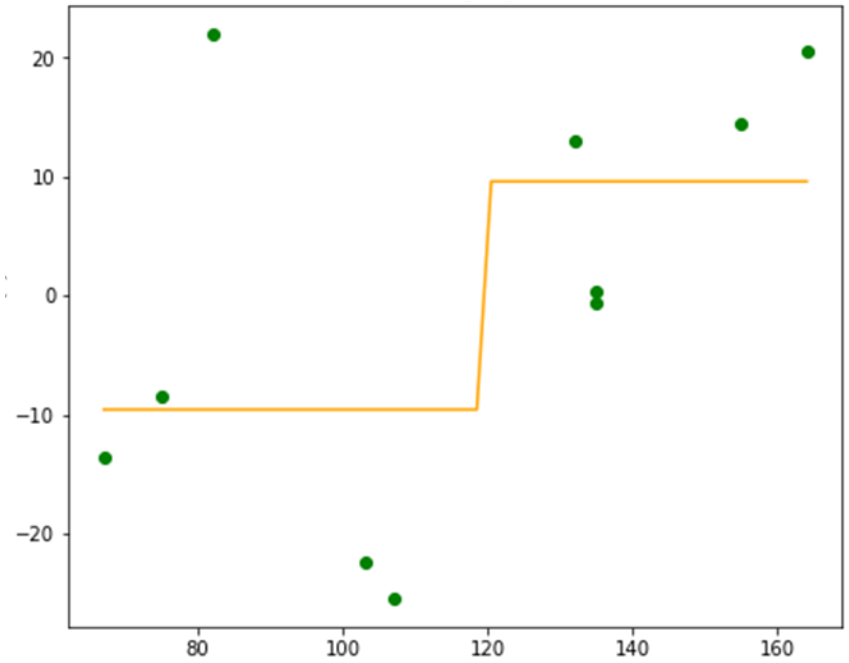
Gradient Boosting – example



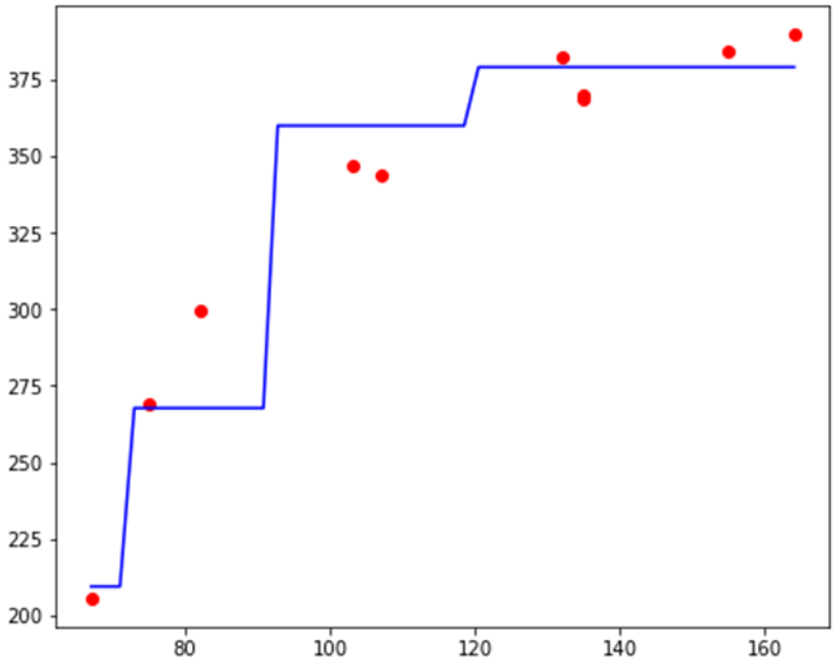


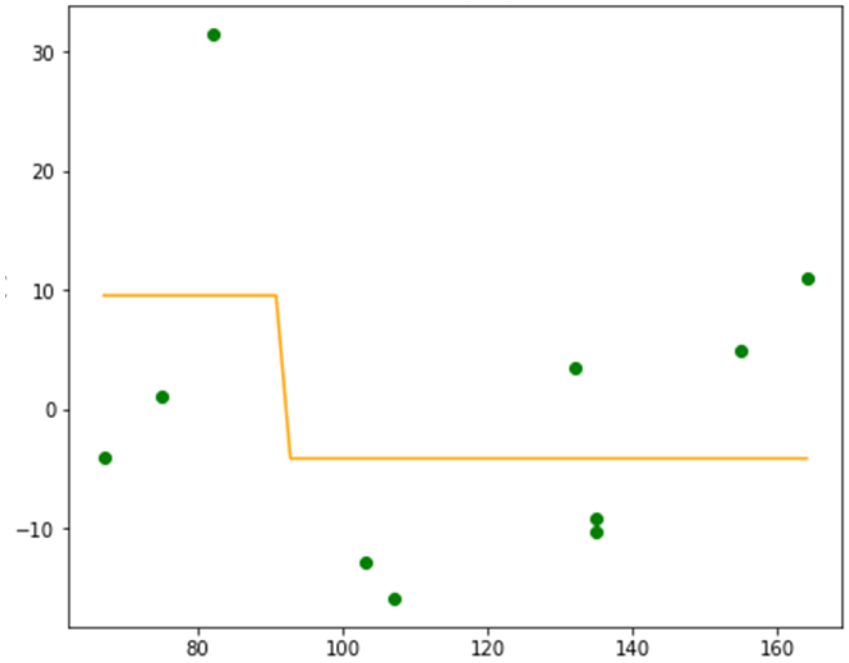
Gradient Boosting – example



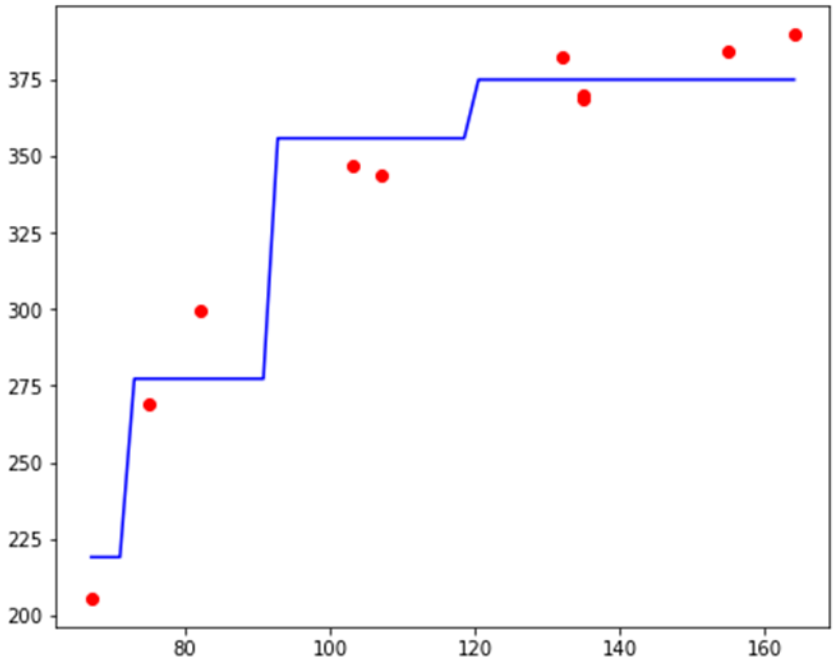


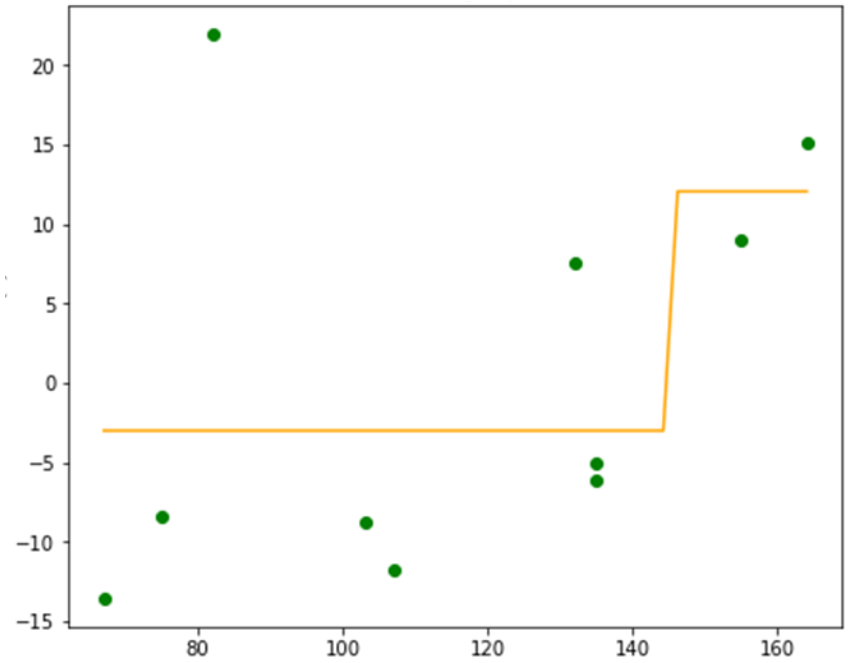
Gradient Boosting – example



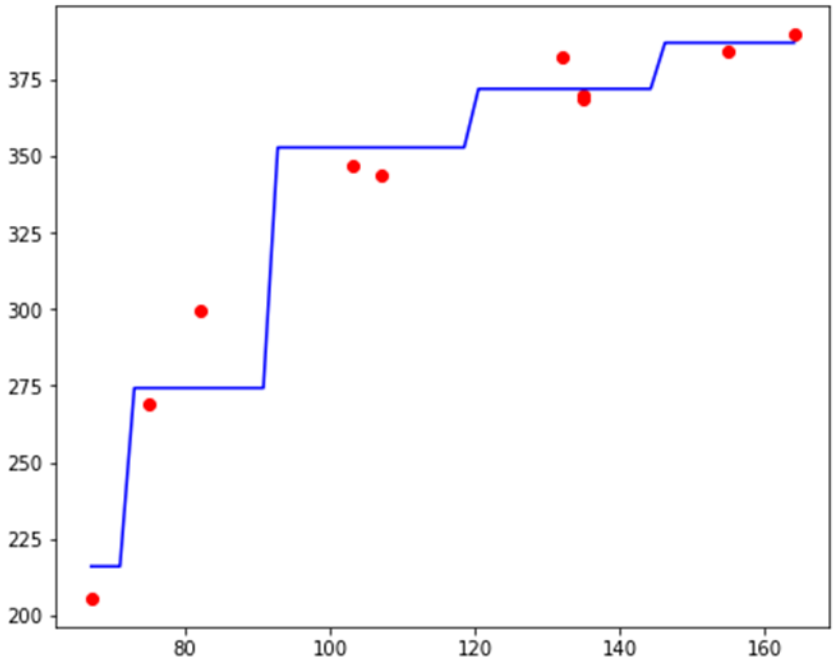


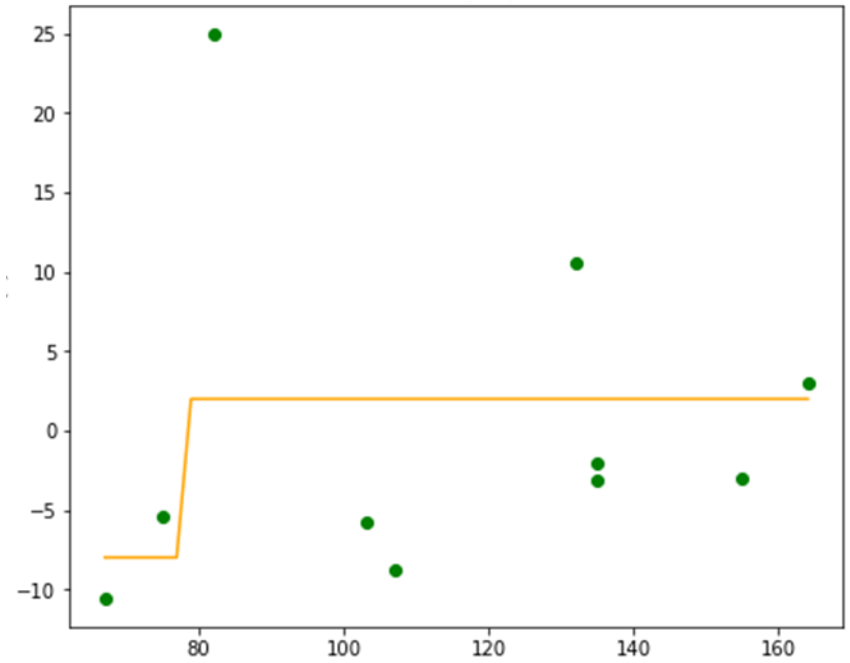
Gradient Boosting – example



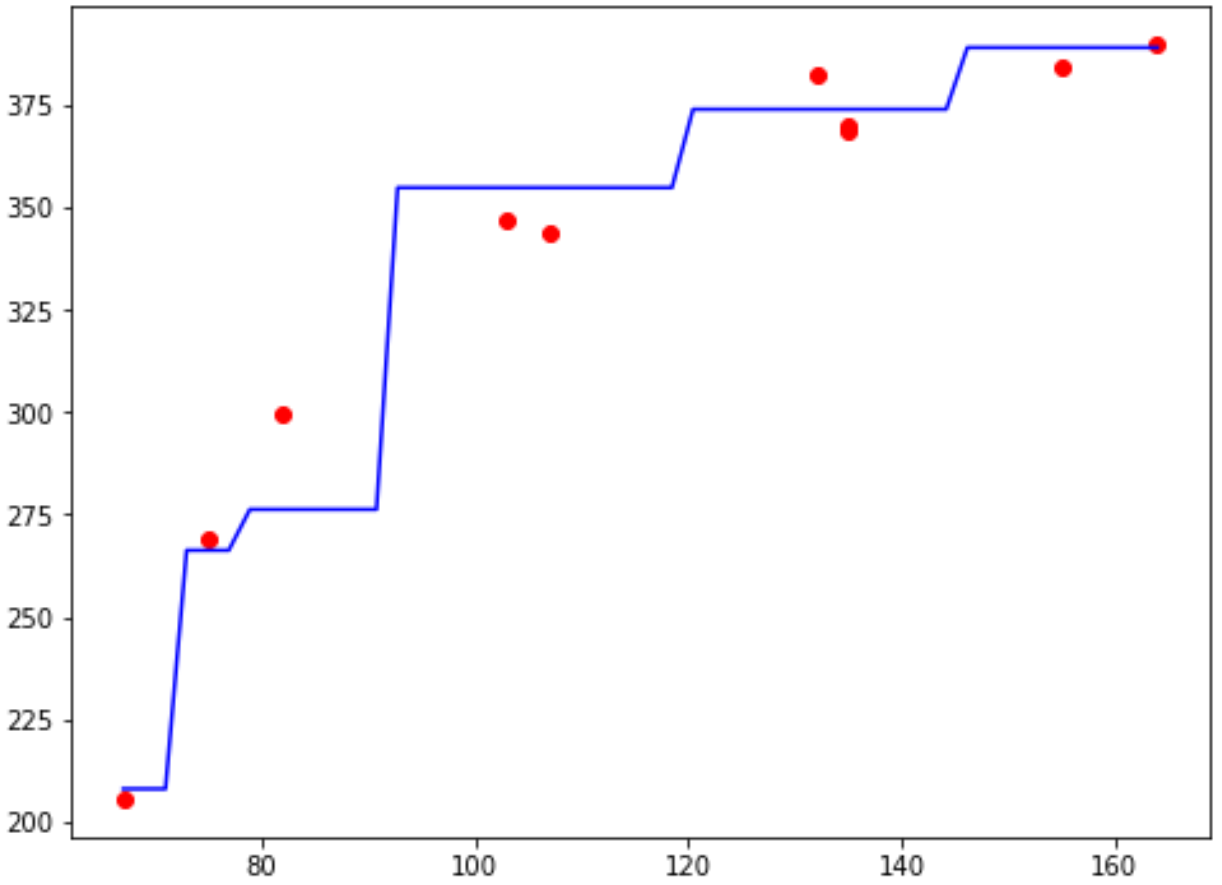


Gradient Boosting – example





Gradient Boosting – example



Classical Models – Ensembles

Other kind of ensembles: Decomposition ensembles

- Goal: To solve a multi-class problem with binary classifiers
- How: Decomposing a multi-class problem into a set of binary problems

- Two ways:
  - OVO (One vs One)
  - OVA (One vs All)

Classical Models – Ensembles

OVO

- Train a classifier for each pair of classes
- Each classifier decides between 2 classes
- Final prediction:
  - Majority vote among all pairwise decisions (or weighted vote)

OVA

- Train a classifier for each class
- Each classifier “class k” vs all other classes
- Final predictions:
  - Pick a class with the highest confidence

Classical Models – Ensembles

Pros/cons OVO

- Simpler and smaller problems
- Computationally faster
- No classifiable region
- Non competent classifiers

Pros/cons OVA

- Uses all the data
- Simpler aggregation of the outputs
- Complex problems (imbalance)
- Computationally more expensive

Outline

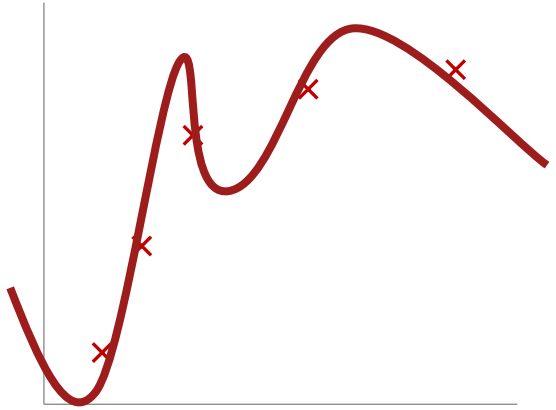
- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Model evaluation and selection

- **Goal:** Assessing how well a model generalizes and selecting the best one
- Three key aspects:
  - Data partitioning
    - Train / validation / test
    - Cross-validation
    - Avoiding data leakage
  - Bias & Variance
    - Underfitting vs. overfitting
    - Generalization gap
    - Trade-offs and remedies
  - Performance metrics
    - Regression, classification and choosing the right metric for the task

Model evaluation and selection – Data partitioning

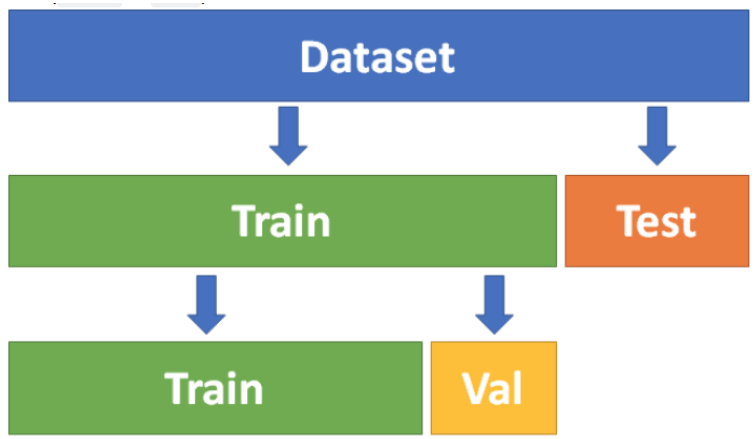
- **Why split the data?**
  - Models can memorize training data
  - We care about the performance on **unseen data** (generalization)
  - We can be fooled by high training accuracy



- **Solution**:
  - Keep some data aside
    - For an unbiased estimate of performance

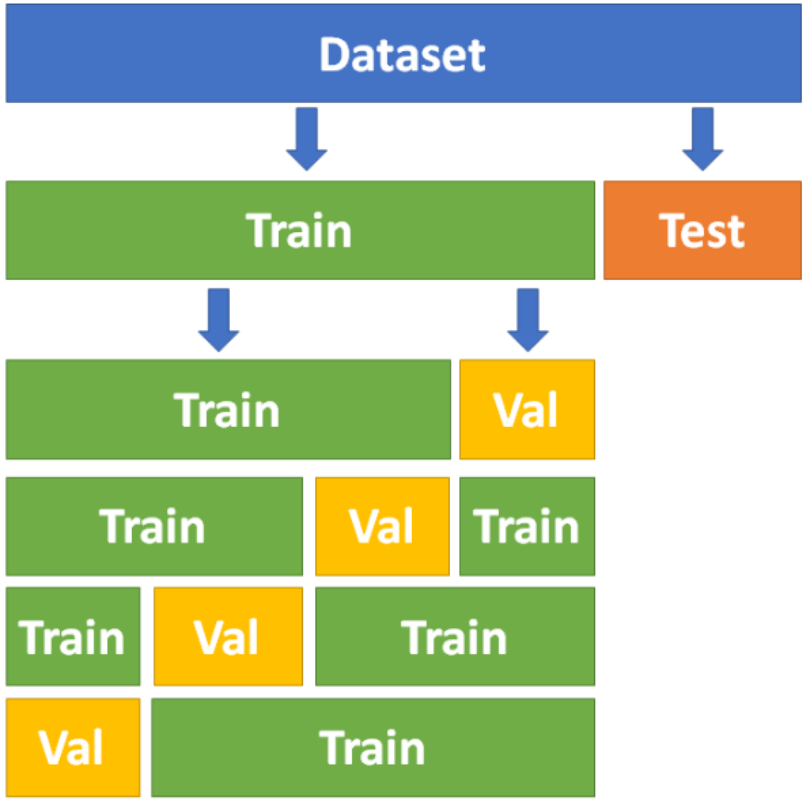
Model evaluation and selection – Data partitioning

- **Train / Validation / Test**
  - Training set
    - To fit model parameters
    - Normally 60 – 80% of the available data
  - Validation set
    - To tune hyperparameters and choose a model
    - Normally 10 – 20%
    - Prevents overfitting to the test set
  - Test set:
    - To estimate final generalization performance
    - Normally 10 – 30%
    - Must remain unseen until the very end



Model evaluation and selection – Data partitioning

- **Cross validation**
  - Problem: A single validation split may be unreliable
    - Depends on luck of partition
  - Solution: Cross-Validation (CV)



- **K-fold CV**: split on k folds, train on k-1, validate on 1; repeat. Then average results
- **Stratified CV**: preserves class proportions in folds
- **Group CV**: respects groups of data
- **Time-series CV**: respects temporal order

Model evaluation and selection – Data partitioning

- **Practical issues**
  - Data leakage
    - Information from test sets leaking into training/validation
    - Produces overly optimistic results
  - Imbalanced datasets:
    - Stratification important
    - May need resampling
  - Large datasets
    - Simple train/test split may be enough (CV expensive)
  - Hyperparameter tuning
    - Use validation (or CV), never test

Model evaluation and selection – Bias & Variance

- **What is bias & variance?**
  - Bias: error from incorrect assumptions in the model (underfitting)
  - Variance: error from excessive sensitivity to training data (overfitting)

- **Why is it important?**
  - Generalization gap: difference between train and test error
    - Training performance different from test performance
  - Understanding bias & variance explains *why models underfit or overfit*

Model evaluation and selection – Bias & Variance

- **Bias**
  - Systematic error due to overly simple assumptions
  - Symptoms:
    - High error on both training and validation sets
    - Model predictions consistently miss the true pattern
  - Detection:
    - Check if training error is already high
  - Remedies:
    - Use a more flexible / complex model
    - Add more informative features
    - Reduce regularization strength

Model evaluation and selection – Bias & Variance

- **Variance**
  - Error due to too much sensitivity to training data
  - Symptoms:
    - Low training error but high validation error
    - Model predictions fluctuate strongly with small data changes
  - Detection:
    - Large generalization gap (train << test error)
  - Remedies:
    - Get more training data
    - Use simpler models
    - Increase regularization

Bias – Variance tradeoff

- Simple models: high bias, low variance
- Complex models: low bias, high variance
- **Goal**: balance bias and variance to minimize total error

Model evaluation and selection – Performance metrics

- **How do we define "good performance"?**
  - Performance metrics
    - They guide model comparison and selection
    - Wrong metric choice can lead to wrong model decision
    - Different tasks, different metrics.
      - Precision
      - Velocity
      - Robustness
      - Scalability
      - Interpretability
      - Model complexity…

Model evaluation and selection – Performance metrics

- **Regression metrics**
  - Mean Squared Error (MSE)
    - Average squared difference between predictions and true values
    - Strongly penalizes large error

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

  - Mean Absolute Error (MAE)
    - Average absolute deviation of predictions from true values
    - More robust to outliers than MSE

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

  - R^2
    - Proportion of variance in the data explained by the model
    - $R^2=1$, perfect fit; $R^2=0$, no improvement over mean predictor
    - Can be negative if the model is worse than the mean predictor

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Model evaluation and selection – Performance metrics

- **Classification metrics (basics)**
  - Accuracy: Proportion of correct predictions w.r.t total
    - Misleading with class imbalance
  - Confusion matrix: counts of TP, FP, FN, TN

		Predicted	
		Positive (1)	Negative (0)
Real	Positive (1)	TP	FN
	Negative (0)	FP	TN

- Precision = TP / (TP + FP)
- Recall (TPR) = TP / (TP + FN)
- Specificity (TNR) = TN / (TN + FP)
- FPR = FP / (FP + TN)
- FNR = FN / (TP + FN)

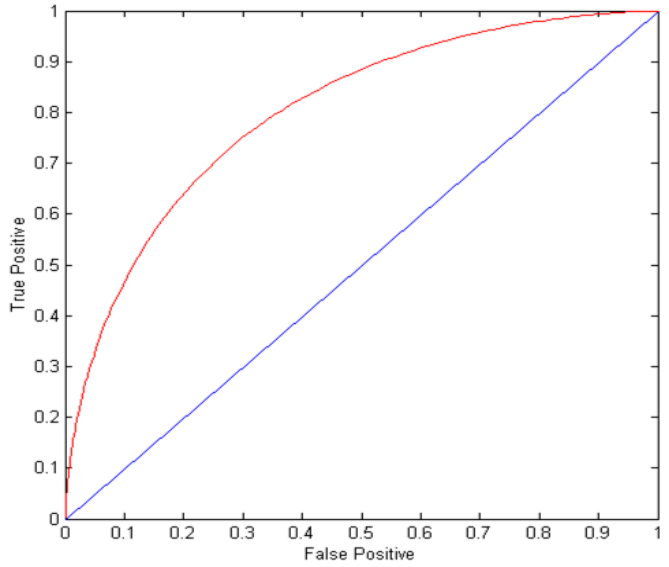
Model evaluation and selection – Performance metrics

- **Classification metrics (balanced)**
  - Balanced accuracy = (TPR + TNR) / 2
  - Geometric mean = sqrt(TPR * TNR)
  - F1 score = 2 * (Precision * Recall) / (Precision + Recall)

		Predicted	
		Positive (1)	Negative (0)
Real	Positive (1)	TP	FN
	Negative (0)	FP	TN

Model evaluation and selection – Performance metrics

- **Classification metrics (curves)**
  - ROC curve (Receiver Operating Characteristic)
    - Plot of TPR vs. FPR as classification threshold varies
    - Diagonal: Random classifier
    - (0,0): all examples classified as negative
    - (1,1): all examples classified as positive
    - (0,1): perfect classifier



  - AUC (Area Under the Curve)
    - Probability that the classifier ranks a random positive higher than a random negative
    - AUC = 0.5 (random guessing), AUC = 1.0 (perfect classifier)

Model evaluation and selection – Performance metrics

- **Classification metrics (curves)**
  - ROC curve (Receiver Operating Characteristic)

Patient	Label	Predicted prob. Of being ill
1	Ill	0,9
2	Ill	0,8
3	Healthy	0,6
4	Ill	0,4
5	Healthy	0,3
6	Healthy	0,1

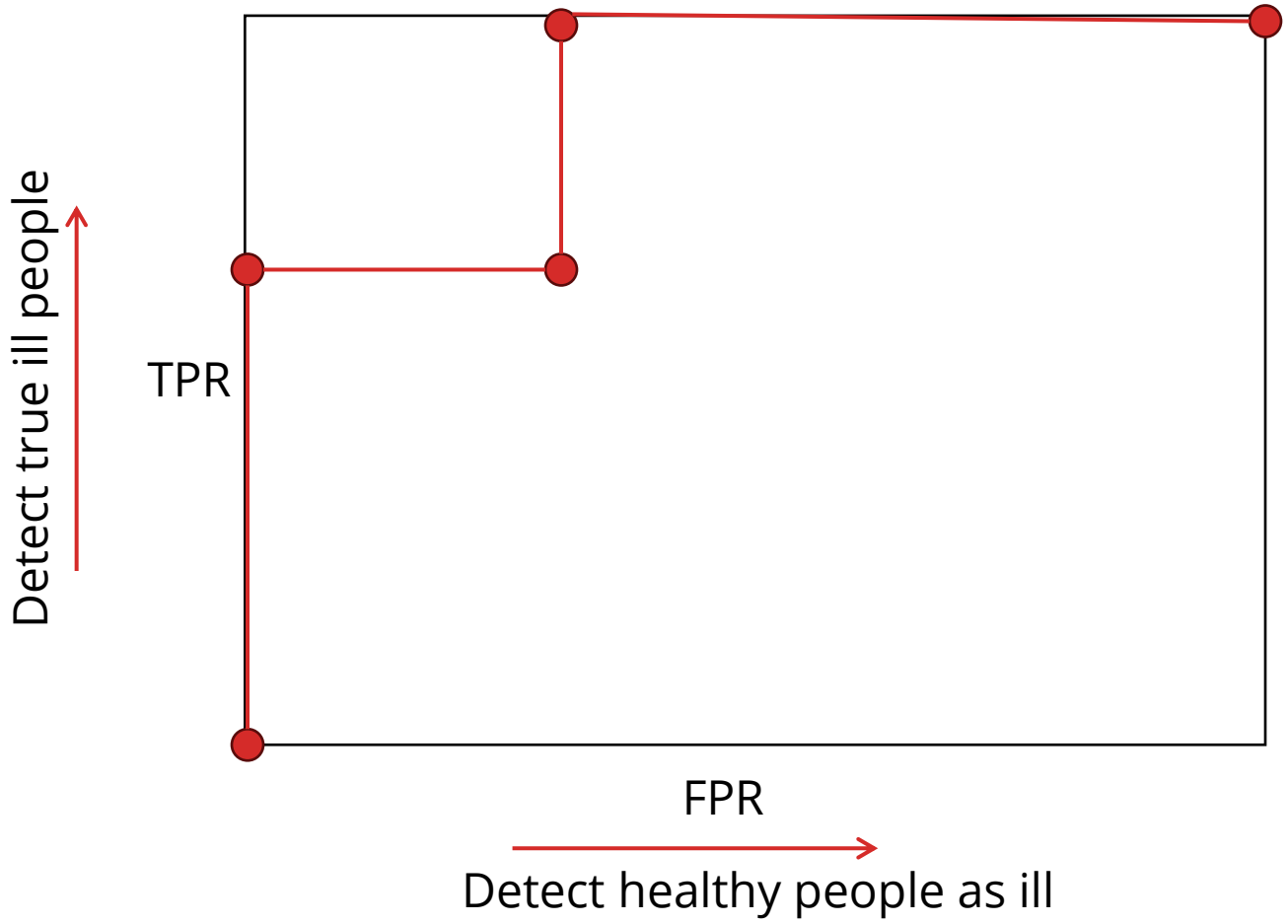
TPR → Proportion of ill people detected
FPR → Proportion of healthy people detected as ill

Threshold of probability	TPR	FPR
1	0/3	0/3
0,75	2/3	0/3
0,50	2/3	1/3
0,35	3/3	1/3
0 (all predicted ill)	3/3	3/3

Model evaluation and selection – Performance metrics

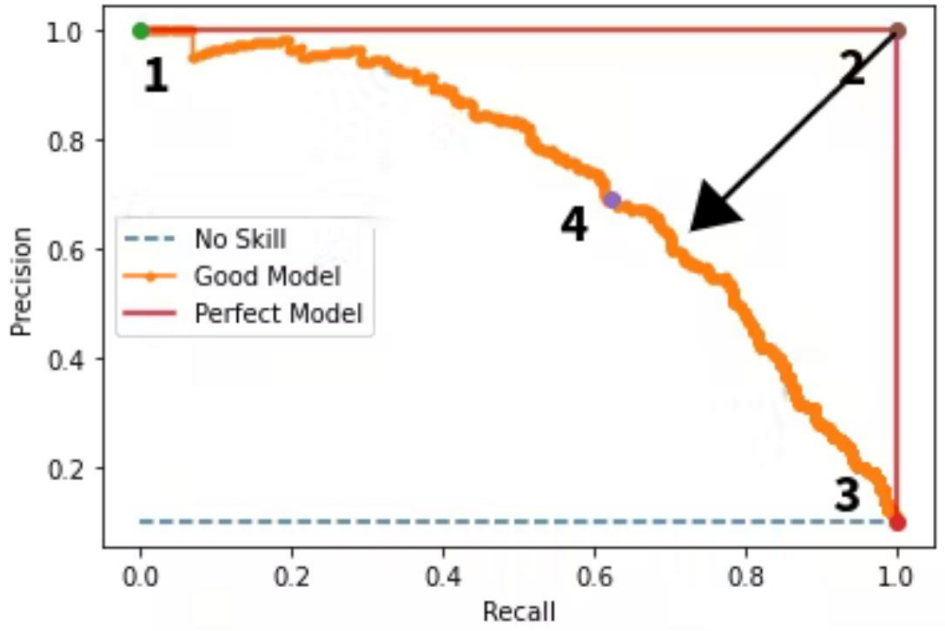
- **Classification metrics (curves)**
  - ROC curve (Receiver Operating Characteristic)

Threshold of probability	TPR	FPR
1	0/3	0/3
0,75	2/3	0/3
0,50	2/3	1/3
0,35	3/3	1/3
0 (all predicted ill)	3/3	3/3



Model evaluation and selection – Performance metrics

- **Classification metrics (curves)**
  - Precision-Recall curve (PR curve)
    - Plot of Precision vs. Recall at different thresholds
    - More informative than ROC for imbalanced datasets
      - Does not use TN



Model evaluation and selection – Performance metrics

- **Multi-class classification metrics**
  - Accuracy: Proportion of correct predictions w.r.t total
    - Misleading with class imbalance
  - Confusion matrix:

Real \ Predicted	C1	C2	C3	C4
C1	h_{11}	h_{12}	h_{13}	h_{14}
C2	h_{21}	h_{22}	h_{23}	h_{24}
C3	h_{31}	h_{32}	h_{33}	h_{34}
C4	h_{41}	h_{42}	h_{43}	h_{44}

  - Precision per class
    - $Prec(C_i) = \frac{\#Hits\ C_i}{\#Examples\ classified\ as\ C_i}$
  - Recall per class
    - $Rec(C_i) = \frac{\#Hits\ C_i}{\#Examples\ of\ class\ C_i}$
  - F1 score per class
    - $Fscore(C_i) = 2\frac{Rec(C_i) * Prec(C_i)}{Rec(C_i) + Prec(C_i)}$

Model evaluation and selection – Performance metrics

- **Multi-class classification metrics**
  - Macro average
    - Compute metric per class, then average equally
    - Macro-precision, macro-recall, macro-fscore
  - Weighted average
    - Compute metric per class, then average by class size
    - Weighted-precision, weighted-recall, weighted-fscore
  - Micro average
    - Aggregate counts (TP, FP, FN) across all classes
    - Coincides with accuracy